

Design and Implementation of a 4-Floor Elevator Controller Using Verilog HDL

1. Introduction

An elevator controller is a critical hardware component used to manage the safe and efficient routing of a multi-floor transport system, handling simultaneous passenger requests from various locations. Unlike a basic sequential servicer that might erratically bounce between floors, a modern controller employs the SCAN algorithm to sweep in one direction until all requests in that path are fulfilled before reversing. This project presents the design and implementation of a synchronous 4-floor Elevator Controller using Verilog HDL. The controller utilizes a Finite State Machine (FSM) to direct the system, asynchronous request latching for memory retention, and combinational lookahead logic to evaluate paths. The design's robustness, including its defense against the "held button" phenomenon, was verified through rigorous waveform simulation.

2. Objectives

The primary objectives of this project are:

- Design a synchronous 4-floor Elevator Controller.
- Implement the hardware logic using Verilog HDL.
- Use a Finite State Machine (FSM) to control the physical routing process.
- Ensure accurate, asynchronous latching of simultaneous passenger requests.
- Implement the SCAN algorithm to prevent erratic directional shifts.
- Verify the design through behavioral waveform simulation.

3. System Overview

The controller receives input signals from four separate floor buttons. Whenever one or multiple buttons are pressed, the controller evaluates its current position and determines the optimal path based on its overarching directional sweep.

The controller halts at requested floors, triggering a synchronous door-open delay, before clearing the specific floor request and resuming its journey. The design consists of three major functional blocks:

- Finite State Machine (FSM)
- Asynchronous Request Latching Logic
- Lookahead Directional Logic

Together, these blocks perform request evaluation, precise motor control, and safe floor servicing.

4. Input and Output Signals

Inputs

- **CLK:** System clock
- **RST:** Active-high synchronous reset
- **REQ[3:0]:** 4-bit bus representing asynchronous button presses for Floors 3, 2, 1, and 0.

Outputs

- **CURRENT_FLOOR[1:0]:** 2-bit encoded output indicating the elevator's physical location.

- **DOOR_OPEN:** Active-high signal indicating doors are open for passenger transfer.
- **DIR[1:0]:** Motor control indicating the current state (00 = Stop, 01 = Up, 10 = Down).

5. Finite State Machine Design

The system is controlled using a three-state Finite State Machine.

IDLE

The controller defaults to the IDLE state to make routing decisions. It first checks if the current floor has an active request; if so, it immediately transitions to the DOOR state. If not, it evaluates the lookahead logic to either transition to MOVING to continue its sweep or reverse its master direction if the path is clear.

MOVING

In this state, the system asserts the motor direction and physically increments or decrements the floor counter. Crucially, it only remains in this state for one clock cycle per floor movement before dropping back to IDLE. This allows the system to dynamically check for "on-the-way" requests that may have been pressed during transit.

DOOR

During the DOOR state, the motor is stopped, and the doors are held open. The controller utilizes a countdown timer to maintain the open state for several clock cycles, allowing time for simulated passenger transfer. Once the timer reaches zero, the specific floor request is cleared, and the FSM returns to the IDLE state.

6. SCAN Algorithm Logic

The routing logic is responsible for selecting the next action based on the current directional priority. Unlike fixed-priority servicing, the SCAN algorithm ensures the elevator sweeps upward until all upper requests are cleared, and then sweeps downward. The controller uses two combinational logic wires (req_above and req_below) that constantly evaluate the latched requests against the current_floor. This logic intercepts requests dynamically, stopping at Floor 2 on the way to Floor 3 if a passenger presses the button mid-journey.

7. Request Latching Logic

A dedicated register stores the currently requested floors. Because passengers press buttons asynchronously, the latched_req memory continuously captures any high bit on the REQ bus. This prevents the system from missing momentary button presses. The request is only dynamically cleared using an inverted bitmask on the exact clock cycle the doors close, guaranteeing the system does not accidentally erase simultaneous requests generated on other floors.

9. Simulation and Verification

Behavioral simulation was performed to analyze the RTL waveforms. The following scenarios were verified:

- Reset operation
- Single requester
- Multiple simultaneous requests
- Dynamic interception of mid-transit requests.
- The "Held Button" phenomenon (simulating continuous bus occupancy).

Simulation waveforms confirmed that:

- The FSM successfully maintained directional sweeps.
- Continuous button holds accurately triggered secondary service cycles without causing state

lockups.

- Bounds-checking prevented floor counter overflow.

10. Results

Simulation results demonstrate that the implemented controller successfully performs safe routing allocation among competing floor requests. The FSM correctly manages the transit process while the combinational lookahead mechanism ensures efficient and logical pathing. The design satisfies the primary objectives of continuous asynchronous latching, correct motor control, and reliable state transitions.

11. Applications

Elevator Controllers using this routing logic are commonly used in:

- Multi-story commercial and residential buildings
- Automated parking garages
- Industrial material handling and sorting facilities
- Robated inventory retrieval systems

12. Advantages

- Prevents starvation for boundary floors.
- Ensures efficient pathing via the SCAN algorithm.
- Simple, synthesizable hardware implementation.
- Defensive bounds checking prevents hardware damage.
- Suitable for synchronous digital systems.

13. Conclusion

This project successfully implemented a synchronous 4-floor Elevator Controller using Verilog HDL. A Finite State Machine was employed to manage the motor operations, while combinational lookahead logic ensured highly efficient pathing using the SCAN algorithm. Simulation verified the correct operation of the design under various rigorous request scenarios, including simultaneous requests and continuous bus holds. The project demonstrates the practical application of digital design concepts such as FSM architecture, asynchronous logic capture, state-boundary protection, and hardware verification.

Tools and Technologies Used

- **Hardware Description Language:** Verilog HDL
- **Design Methodology:** Register Transfer Level (RTL) Design
- **Verification:** Behavioral Waveform Simulation
- **Architecture:** Finite State Machine (FSM)